

Documentación Técnica - Configuración y Manejo de Archivos

Proyecto 1 – Manejo e implementación de archivos

1. Formato de Almacenamiento: JSON vs. Alternativas

Para la gestión de la configuración de la aplicación, se seleccionó el formato JSON, descartando alternativas como XML o INI, basándonos en los siguientes criterios técnicos:

- **Facilidad:** JSON permite mapear los datos de configuración de forma nativa a diccionarios en Python mediante la librería estándar json, que a diferencia del formato INI o XML, JSON preserva los tipos de datos de forma implícita.
- **Legibilidad y tamaño:** JSON utiliza una sintaxis minimalista basada en pares clave-valor, lo que reduce drásticamente el peso del archivo en comparación con la verbosidad de las etiquetas de apertura y cierre de XML.
- **Robustez ante corrupción:** JSON posee una sintaxis estricta, si el archivo sufre alteraciones indebidas, el analizador de Python lanza inmediatamente una excepción estructurada, lo que permite atrapar el error antes de inyectar variables corruptas a la lógica de la aplicación.

2. Implementación de la escritura segura y el respaldo (Backup)

Para garantizar que la configuración no se corrompa ante un cierre inesperado o fallo del sistema, se implementaron dos mecanismos de seguridad en la clase:

1. **Generación del Respaldo (.bak):** Antes de sobrescribir los datos, el sistema verifica si ya existe un archivo de configuración anterior. Si existe, abre el archivo original en modo lectura, extrae su contenido completo y lo escribe en un nuevo archivo llamado config.bak. Esto permite recuperar la última configuración funcional en caso de fallo.
2. **Escritura Atómica (.tmp):** Para guardar los nuevos ajustes, no se sobrescribe el archivo original directamente. Primero, los datos se guardan en un archivo temporal (config.tmp). Una vez que la escritura finaliza con éxito y sin errores de formato, se utiliza la función `os.replace()` para reemplazar el archivo temporal por el definitivo (config.json). Si algo falla durante el proceso, el bloque `except` captura el error y elimina el archivo temporal residual.

```

# noinspection PyBroadException
def guardar_configuracion(self, datos_configuracion): 1 usage (1 dynamic)  ⚡ Melqui Garcia
    """Patrón de escritura segura usando archivo temporal."""
    try:
        if os.path.exists(self.nombre_archivo):
            with open(self.nombre_archivo, "r", encoding="utf-8") as original:
                contenido_anterior = original.read()

            with open(self.archivo_respaldo, "w", encoding="utf-8") as respaldo:
                respaldo.write(contenido_anterior)

        with open(self.archivo_temporal, "w", encoding="utf-8") as temporal:
            json.dump(datos_configuracion, temporal, indent=4, ensure_ascii=False)

        os.replace(self.archivo_temporal, self.nombre_archivo)

        self.configuracion_actual = datos_configuracion.copy()
        return True

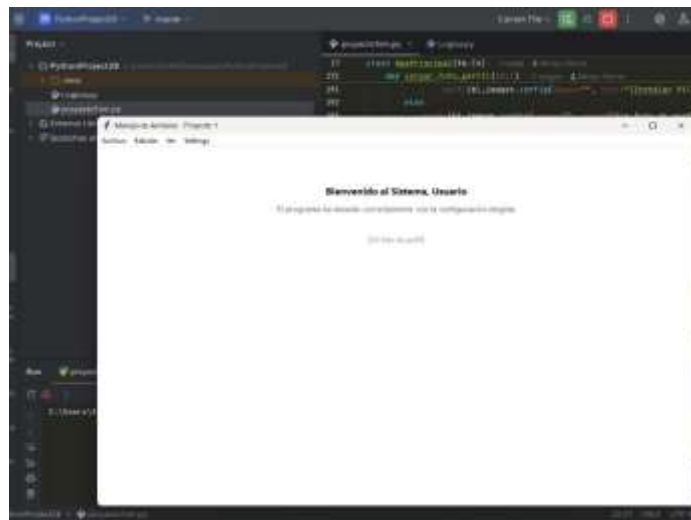
    except Exception as e:
        # Imprime el error para que la variable 'e' sea utilizada y no marque advertencia
        print(f"Fallo crítico al guardar (capturado por seguridad): {e}")
        if os.path.exists(self.archivo_temporal):
            os.remove(self.archivo_temporal)
        return False

```

3. Manejo de Casos de Error

El sistema fue diseñado para no interrumpir su ejecución ("crashear") ante excepciones de entrada/salida, todos los accesos a los archivos se encuentran encapsulados en un bloque try, manejando los tres escenarios críticos exigidos:

1. **Archivo Ausente:** Si el archivo es eliminado o no ha sido creado, el sistema atrapa el error de forma silenciosa e inicializa la aplicación cargando un diccionario en memoria con los valores por defecto (tema claro, fuente 12, idioma español).



2. **Archivo Corrupto (json.JSONDecodeError):** Si el archivo contiene errores de sintaxis o basura inyectada, el sistema aborta la lectura y levanta una interfaz de advertencia para el usuario.



3. **Sin Permisos (PermissionError):** Si el sistema operativo deniega la lectura del archivo, se notifica al usuario mediante un cuadro de diálogo y el programa continúa su ejecución con los valores por defecto.



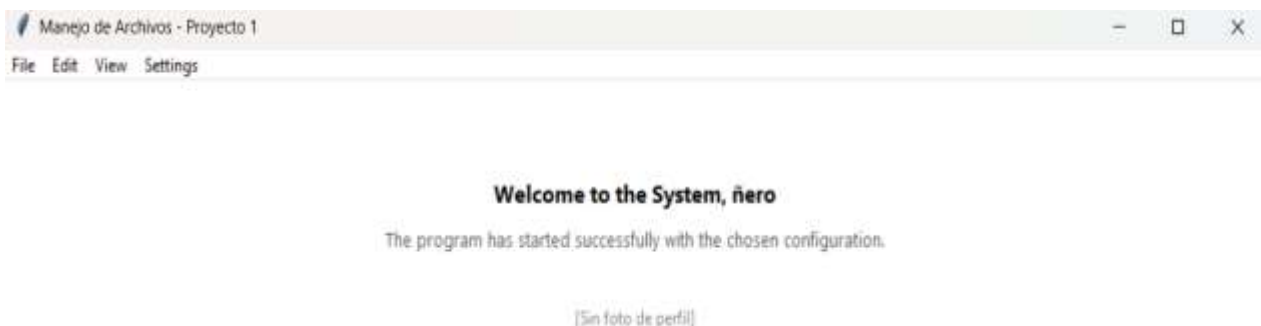
4. Generación de Archivos y Codificación

Para garantizar que caracteres especiales propios del idioma español (como la letra 'ñ' o vocales con tilde) no se deformen al escribirse en el archivo, se configuró explícitamente el parámetro `encoding="utf-8"` en la apertura de los flujos de datos, acompañado del flag `ensure_ascii=False` en el volcado de JSON.

5. Funcionamiento

El sistema cuenta con una interfaz gráfica (GUI) desarrollada con la librería `tkinter`, la cual aplica dinámicamente los ajustes leídos desde el archivo de configuración.

- **Pantalla Principal (Traducción y Parámetros aplicados):**



- **Ventana de Settings:**

view settings

Ventana de Settings

Nombre Usuario:

Rero

Tema Interfaz:

claro

Idioma:

en-US

Tamaño fuente (Número entero):

12

Color de Menú:

#EEEEEE

Color de Letra:

#000000

Foto de Perfil (Ruta):

Examinar...

Cancelar Guardar Configuración